SW407 · SISTEMAS OPERATIVOS · SEMANA 4

Planificación de Procesos

¿A quién atiende primero el cocinero? El arte de ordenar la fila de pedidos

Escuela de Ingeniería de Software

Facultad de Ingeniería Industrial y de Sistemas — UNI

Docente: Mg. Ing. José Carlos, García La Riva · 2026-II

Logros de la sesión

1

Explicar los objetivos de la planificación de procesos

Maximizar el rendimiento del sistema y minimizar el tiempo de espera de los procesos.

2

Aplicar los algoritmos FCFS, SJF, Round Robin y por Prioridad

Sobre un mismo conjunto de procesos, para comparar su comportamiento.

3

Calcular el tiempo de espera y el tiempo de respuesta

Como métricas para evaluar la calidad de un algoritmo de planificación.

4

Simular e implementar algoritmos de planificación

En Python o C, comparando resultados entre distintos enfoques.

EL ESCENARIO

La fila de pedidos ya no cabe en la cocina

CocinaPerú Express se volvió tan popular que, en las horas pico, siempre hay varios pedidos esperando en el estado "Listo" al mismo tiempo. El cocinero termina un plato y levanta la vista: hay cuatro pedidos en la fila. ¿Cuál prepara ahora?

Si los atiende en el orden exacto en que llegaron, un pedido grande de banquete puede hacer esperar innecesariamente a tres pedidos pequeños que llegaron después. Si siempre prioriza a los pedidos más rápidos, el pedido grande podría no cocinarse nunca. Y si un cliente VIP llega con un pedido urgente, ¿debería saltarse toda la fila?

Esa decisión —cuál pedido pasa primero a la parrilla cuando hay varios esperando— es exactamente el trabajo del PLANIFICADOR (scheduler) del sistema operativo. Hoy conoceremos cuatro estrategias distintas que usa para decidirlo, y descubriremos que no existe una única forma "correcta" de ordenar la fila.

¿Qué es la planificación de procesos?

Es la función del sistema operativo que decide, en cada momento, cuál de los procesos en estado "Listo" recibe la CPU a continuación.

1. **Maximizar el rendimiento**
   Aprovechar al máximo la CPU (mantenerla ocupada) y atender la mayor cantidad de procesos posible por unidad de tiempo (throughput).

2. **Minimizar el tiempo de espera**
   Reducir cuánto tiempo pasa cada proceso en la fila de "Listo" antes de que le toque su turno en la CPU.

3. **Un objetivo con tensiones**
   Como veremos, mejorar una métrica (p. ej. la espera promedio) puede empeorar otra (p. ej. qué tan rápido responde el sistema a un proceso nuevo).

Cómo medimos un algoritmo de planificación

Tiempo de espera (waiting time)

Cuánto tiempo estuvo el pedido en la fila de "Listo" antes de empezar a cocinarse. Fórmula: tiempo de inicio de ejecución − tiempo de llegada.

Tiempo de respuesta / retorno (turnaround time)

Cuánto tiempo pasó desde que el pedido llegó hasta que quedó completamente servido. Fórmula: tiempo de finalización − tiempo de llegada.

Promedios del sistema

Un algoritmo se evalúa comparando el promedio de estas métricas sobre todos los procesos, no el resultado de un solo caso.

CASO DE ESTUDIO

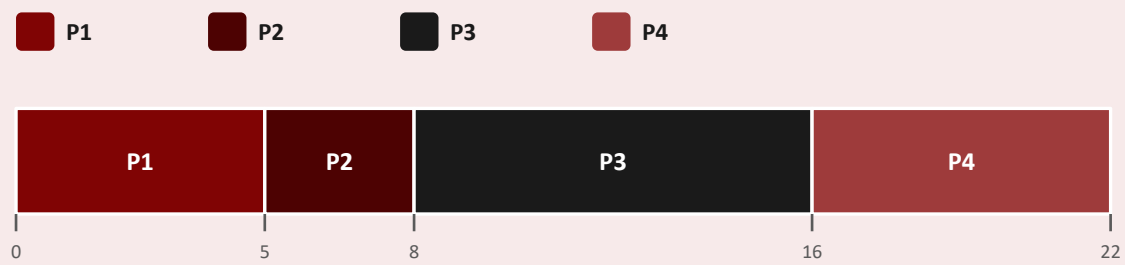
Cuatro pedidos, cuatro decisiones distintas

Usaremos este mismo conjunto de pedidos para comparar los cuatro algoritmos de planificación de esta sesión.

Pedido	Llegada	Duración (preparación)	Prioridad (1 = más alta)
P1	t = 0	5 minutos	3
P2	t = 1	3 minutos	4 (la más baja)
P3	t = 2	8 minutos	1 (la más alta)
P4	t = 3	6 minutos	2

ALGORITMO 1

FCFS — Primero en llegar, primero en ser atendido



Orden de atención: exactamente el orden de llegada a la fila.

Espera por pedido (min): P1: 0 P2: 4 P3: 6 P4: 13

Espera promedio: 5.75 minutos

- **Cómo funciona**

  El planificador atiende los pedidos exactamente en el orden en que llegaron a la fila, sin excepciones.

- **Ventaja**

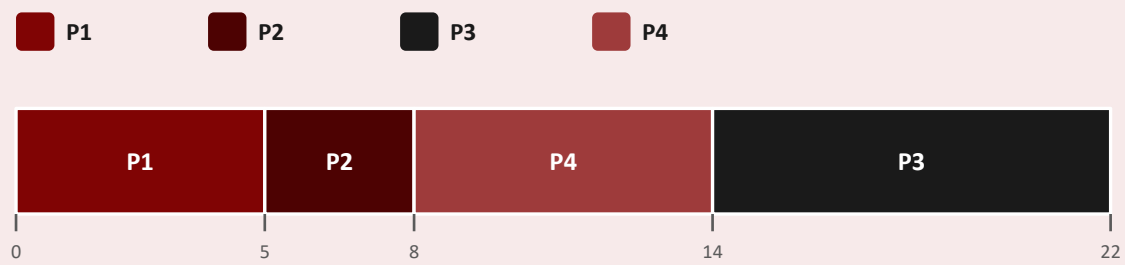
  Simple de implementar y percibido como "justo" en apariencia.

- **Problema: efecto convoy**

  Si P3 (8 minutos) hubiera llegado primero, todos los pedidos más cortos detrás de él esperarían innecesariamente.

ALGORITMO 2

SJF — El trabajo más corto primero



De los pedidos ya en fila, siempre se elige el de menor duración.

Espera por pedido (min): P1: 0 P2: 4 P4: 5 P3: 12

Espera promedio: 5.25 minutos

- **Cómo funciona**

  En cada oportunidad, el planificador revisa los pedidos ya en la fila y elige el de menor tiempo de preparación.

- **Ventaja**

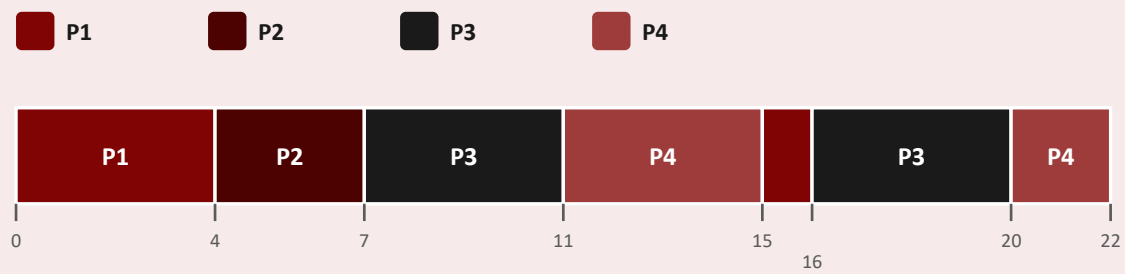
  Es matemáticamente óptimo para minimizar el tiempo de espera promedio (el mejor resultado posible: 5.25 min).

- **Problema: inanición (starvation)**

  P3 (el pedido grande) puede quedar esperando indefinidamente si siempre llegan pedidos más cortos que él.

ALGORITMO 3

Round Robin — Turnos rotativos por cuanto de tiempo



Cuanto (quantum) = 4 min. Cada pedido recibe un turno fijo; si no termina, vuelve al final de la fila.

Espera por pedido (min): P1: 11 P2: 3 P3: 10 P4: 13

Espera promedio: 9.25 min · Respuesta promedio: 4.0 min

(cada pedido empieza a cocinarse mucho antes, aunque termine más tarde)

- **Cómo funciona**

  Cada pedido recibe un turno fijo (quantum) en la parrilla; si no termina, se pone al final de la fila y sigue el siguiente.

- **Ventaja**

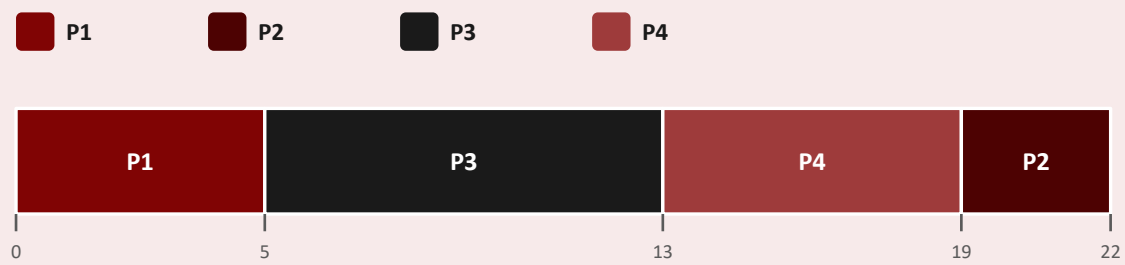
  Tiempo de respuesta muy parejo: ningún pedido espera mucho para que lo empiecen a atender (4.0 min en promedio).

- **Costo**

  Más cambios de contexto entre pedidos, y la espera promedio total puede ser mayor que en SJF (9.25 min en este caso).

ALGORITMO 4

Prioridad — El cliente VIP pasa primero



P3 tiene la prioridad más alta (1) y pasa antes que P2 y P4, aunque haya llegado después.

Espera por pedido (min): P1: 0 P3: 3 P4: 10 P2: 18

Espera promedio: 7.75 minutos

- **Cómo funciona**

  A cada pedido se le asigna una prioridad; el planificador siempre atiende primero al de mayor prioridad, sin importar su duración.

- **Efecto visible aquí**

  P2 es el pedido más corto (3 min) pero tiene la prioridad más baja, y termina esperando 18 minutos: casi 6 veces su propia duración.

- **Solución: envejecimiento (aging)**

  La prioridad de un pedido va subiendo cuanto más tiempo lleva esperando, para evitar que quede en inanición indefinidamente.

Comparación de resultados sobre el mismo caso

Algoritmo	Espera promedio	Respuesta promedio	Riesgo principal
FCFS	5.75 min	5.75 min	Efecto convoy si un pedido grande llega primero
SJF	5.25 min (óptimo)	5.25 min	Inanición de pedidos grandes
Round Robin	9.25 min	4.0 min (el mejor)	Más cambios de contexto; espera total más alta
Prioridad	7.75 min	7.75 min	Inanición de pedidos de baja prioridad

Simulando FCFS y SJF en Python

python

```
pedidos = {"P1": (0, 5), "P2": (1, 3),
           "P3": (2, 8), "P4": (3, 6)}
# formato: (llegada, duración)

def sjf(pedidos):
    restantes, t, esperas = dict(pedidos), 0, {}
    while restantes:
        listos = {p: d for p, (l, d) in restantes.items()
                  if l <= t}
        if not listos:
            t = min(l for l, d in restantes.values())
            continue
        p = min(listos, key=listos.get)
        esperas[p] = t - restantes[p][0]
        t += restantes[p][1]
        del restantes[p]
    return esperas
```

La misma lógica, cambiando solo el criterio de selección (min(...)), permite comparar SJF frente a Prioridad.

- **Idea central**

  En cada vuelta del ciclo, se filtran los pedidos ya "listos" (llegada ≤ t) y se elige uno según el criterio del algoritmo.

- **Reutilizable**

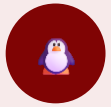
  Cambiando min(listos, key=duración) por key=prioridad se obtiene el algoritmo de Prioridad con la misma estructura.

- **Verificación**

  Los resultados de este código coinciden con los tiempos de espera mostrados en las láminas anteriores (5.25 min en promedio para SJF).

APLICACIÓN

¿Dónde vemos planificación de procesos hoy?



Linux — CFS (Completely Fair Scheduler)

Reparte el tiempo de CPU de forma proporcional entre procesos, evitando tanto el efecto convoy como la inanición.



Sistemas de tiempo real

Usan planificación por prioridad estricta: una alarma de seguridad no puede esperar a que termine una tarea de bajo valor.



Sistemas de escritorio

Usan variantes de Round Robin con prioridades dinámicas para mantener la interfaz gráfica siempre receptiva.



Servidores y colas de trabajos

Aplican SJF o variantes para procesar primero las tareas cortas y mantener bajo el tiempo de espera promedio de los usuarios.

TALLER EN CLASE

Simula y compara los algoritmos de planificación

Trabaja en pareja. Usa el mismo conjunto de pedidos (P1-P4) del caso de estudio de esta sesión.

01 Implementa en Python o C los algoritmos FCFS, SJF, Round Robin (quantum = 4) y Prioridad.

02 Verifica que tu programa reproduzca los mismos tiempos de espera mostrados en las láminas (FCFS: 5.75, SJF: 5.25, RR: 9.25, Prioridad: 7.75).

03 Agrega un quinto conjunto de pedidos propuesto por tu equipo y compara los cuatro algoritmos sobre ese nuevo caso.

04 Documenten en qué escenario recomendarían cada algoritmo y por qué (piensen en tiempo real, servidores, escritorio, etc.).

Conclusiones

1. No existe un algoritmo de planificación universalmente mejor: cada uno optimiza una métrica distinta (espera, respuesta, equidad) a costa de otra.

2. SJF minimiza matemáticamente la espera promedio, pero puede dejar en inanición a los procesos largos si no se controla.

3. Round Robin sacrifica espera total por un tiempo de respuesta mucho más parejo, clave en sistemas interactivos.

4. La prioridad permite atender lo más importante primero, pero requiere mecanismos como el envejecimiento para no perjudicar injustamente a procesos de baja prioridad.

Referencias bibliográficas

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley — Capítulo 5 (CPU Scheduling).

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson — Sección de planificación de procesos.

Stallings, W. (2018). Operating Systems: Internals and Design Principles (9th ed.). Pearson.

The Linux Kernel documentation (2026). CFS Scheduler. kernel.org/doc/html/latest/scheduler/sched-design-CFS.html

Universidad Nacional de Ingeniería (2026). Sílabo SW407 — Sistemas Operativos, Escuela de Ingeniería de Software.